Neither Layer Alone: Epistemic Integrity Requires Hierarchical Joint Design for Long-Running AI Agents

Zhihong Shen
Microsoft
Redmond, WA 98052
zhihosh@microsoft.com

Abstract

Long-running AI agents fail not only when inference fails or tools are underspecified, but when independently evolving model and harness layers change the semantics of belief, capability, and goal commitments across their boundary — a failure class this paper terms *interface volatility*. This paper argues that *epistemic integrity* must be treated as a first-class architectural constraint, achievable only through joint model–harness design organized around an explicit interface contract. The contribution is a four-level operational hierarchy — goal validity, archetype sequencing, tool-instance selection, and invocation-level failure discrimination — that specifies what the boundary must preserve and what the model must return for those guarantees to compose. This reframes long-running agent design away from flat action loops and toward contract-preserving control over persistent state. Evaluation and training should therefore derive from the contract itself, testing whether belief, tool, and goal commitments hold across session boundaries and independent layer upgrades.

1 Introduction

A model upgrade can degrade an agent's behavior even when its memory store, tool schemas, and goal records are unchanged. The model need not be defective; the harness may simply have been built against a prior model prior. It encoded assumptions about when the model would trust memory, defer to tool output, preserve goals, or allow external state to override its own beliefs. When the model prior changes, those assumptions can break without any schema, tool, or record changing. This exposes an architectural asymmetry: the model supplies the dominant knowledge prior, while the harness remains reactive, forced to compensate after each model shift. The problem is not ordinary nondeterminism but the loss of stable epistemic meaning under supposedly compatible updates. This paper calls this failure *interface volatility*: behavioral degradation caused by a change in one layer that becomes visible only through its integration with the other.

The usual response is to improve one side of the stack in isolation. Better models are expected to interpret retrieved beliefs, registered tools, and persisted goals more reliably. Better harnesses are expected to provide cleaner schemas, stronger validation, and more deterministic orchestration. But neither improvement specifies the boundary condition between them. The model still assigns semantic interpretation to external state, while the harness still persists state whose operational force depends on how a particular model reads, prioritizes, or overrides it. Benchmarks may catch regressions, and harnesses may block unsafe actions, but neither guarantees that knowledge, capability, and commitment remain meaningful across independent model and harness changes. As a result, training can reward successful task completion while leaving the more granular internal state required for safe resumption unspecified, unversioned, and untested.

Preprint.

This paper argues that long-running agents require *epistemic integrity* as a first-class architectural constraint, achievable only through hierarchical joint design of model and harness, with an explicit model-harness interface contract that precedes and determines the training objective. Stated in the falsifiable form: a long-running agent system whose model-harness interface does not carry epistemic metadata across sessions cannot maintain epistemic integrity in the long-running regime, regardless of model capability or harness sophistication.

The argument unfolds through three linked claims. First, failures in memory, tool use, and planning should not be treated as separate defects, but as surface forms of a deeper boundary challenge between model and harness: the challenge of **knowing, doing, and deciding** under distributed state — where *knowing* concerns the representation and priority of memory, context, internal state, and world state; *doing* concerns capability invocation and accumulated effects, and *deciding* concerns goal validity and commitment over time. Second, it proposes *epistemic integrity* as an architectural principle: the authority of external memory, episodic context, tool outputs, goal records, and in-model priors must be governed by explicit, conditional precedence rules rather than implicit prompting conventions. These rules should be specified, learned, versioned, and audited as part of the model–harness contract, with safeguards for stale, inconsistent, or corrupted state. Third, it defines an operational hierarchy in which goal validity, tool and operation archetype sequencing, tool-instance selection, and invocation-level failure discrimination are mediated by distinct interface artifacts and reward signals. The purpose of this hierarchy is to replace flat action-loop thinking with contract-preserving control over persistent, fine-grained state.

This claim is stronger than the now-common observation that the harness matters. Recent work has treated the harness as a first-class reliability determinant rather than a thin wrapper around model inference. This paper accepts that premise but gives it a stricter architectural form: it specifies what the model–harness boundary must preserve, what the model must return for those guarantees to compose, and why this explicit, versioned, model-independent contract is the artifact on which long-running agent evaluation and training must be built. The contribution is therefore not that the harness matters, but that the model–harness interface is the load-bearing object of long-running agent design.

The remainder of the paper develops this argument in five steps. Section 2 defines *Agent Epistemic Integrity* (AEI) and introduces the Knowing, Doing, and Deciding domains. Section 3 argues that long-running agents require evaluation over persisted trajectories, not only session-bounded traces. Section 4 specifies the model–harness contracts required for belief, capability, and goal state. Section 5 derives reward signals and training infrastructure from those contracts. Sections 6 and 7 position the argument relative to prior work and outline the empirical next steps.

2 Agent Epistemic Integrity: A Framework

2.1 The Property

Agent Epistemic Integrity (AEI) is the architectural property that an agent's *active state* remains sufficiently coherent, inspectable, and correctable for the actions it takes over time. The *active state* is structurally essential and operationally central: it includes not just retrieved facts, but also the system's current uncertainty annotations, its understanding of what capabilities have already been invoked and with what effects, as well as its representation of what it is committed to doing next.

AEI is about calibrated operation under incomplete information. A system can be uncertain and maintain AEI only when it behaves proportionally to that uncertainty. Conversely, a system can violate AEI even when individual outputs appear correct, if it acts on stale, partial, or weakly grounded state as if they were fully reliable. Crucially, AEI is a system-level property, not a model-level guarantee. It cannot be ensured by any individual component, and it cannot be evaluated on the basis of individual outputs. Its value is in directing architectural attention toward how the system represents epistemic state and exposes that representation for correction.

2.2 Three Invariant Domains

The problem space of long-running agentic systems decomposes, at the architectural level, into three invariant domains: **Knowing, Doing, and Deciding**. They are invariant because any agentic system must solve a version of each: it must maintain a picture of state, act through available capabilities,

Table 1: The AEI grid. Italicized assumptions are structurally enforced in session-based systems and structurally violated in long-running operation — exposing the architectural primitive each domain newly requires when sessions extend, persist, and span model upgrades.

Domain	**Session-Based**	**Long-Running**	**Primitive Required**
Knowing	Context fresh by session boundary *(freshness assumed)*	Detect staleness; revise superseded beliefs	Belief revision
Doing	Invocations locally bounded *(idempotency assumed)*	Track cumulative state; reason about safe resumption	Capability state management
Deciding	Goal stable for session duration *(goal validity assumed)*	Track goal validity over time; surface intent drift	Goal lifecycle management

and decide which goals or actions remain appropriate. A solution in one domain does not substitute for a solution in another.

Knowing is memory and context management — what the agent holds as true about the world, itself, and its history. The challenge in the long-running regime is not storage but coherence over time: detecting when standing beliefs have been superseded, resolving conflicts across evidence arriving at different times, and tracing the provenance of decisions back to their epistemic basis.

Doing is capability management — what actions the agent can take, under what conditions, and what the cumulative effect of prior invocations has been. In the long-running regime, capability availability is not binary, effects are not idempotent, and degradation is frequently latent. The challenge is not selecting the right tool but tracking what the system has already done with it.

Deciding is planning and deliberation — how the agent selects and sequences actions given its beliefs and capabilities. In the long-running regime, this extends beyond step-level planning to goal lifecycle: whether the goals that initiated a task remain valid, and whether the agent has a surface for human correction before it acts on goals that have silently expired or updated.

These domains are analytical lenses on a single runtime loop, not separable subsystems. At the model layer, Knowing, Doing, and Deciding are simultaneous aspects of each inference step: the model draws on parametric knowledge in its weights, episodic information in the context window, and harness-supplied external state such as retrieved memories, tool specifications, capability records, and goal-state records. Their separation becomes diagnostic only when attributing failure modes, and architectural only at the harness layer, where memory stores, tool executors, goal-state stores, and orchestration logic can be separately implemented. AEI therefore requires an explicit knowledge-priority contract across all three domains: the interface must specify, version, and audit when harness-supplied runtime state, session context, or parametric priors are authoritative, rather than assuming a fixed precedence order among them.

Table 1 organizes the three domains against two deployment paradigms. The session-based column reflects what current architectures implicitly assume — and encodes the assumption each cell silently makes. The long-running column is where those assumptions fail, surfacing new architectural primitives that session-bounded designs do not require and therefore do not provide. These three domains are not independent — AEI's central claim is that their coupling, not any single domain, is the primary failure surface.

2.3 Prospective Memory as the Unifying Primitive

All three long-running primitives — belief revision, capability state management, goal lifecycle management — share a common structural requirement: the agent must maintain a live, queryable representation of its own intended future state. This is the function of prospective memory.

The term is borrowed from cognitive psychology, where prospective memory refers to remembering to perform an intended action at a future time [2]. The engineering construct proposed here is related

in motivation: a first-class, inspectable data structure encoding the validity conditions, provenance, and dependencies of the agent's forward commitments, exposed through a query interface to both the execution engine and the human oversight layer.

Prospective memory is the natural insertion point for human steering. What an operator most needs to inspect and correct is not the agent's past actions — which may have already committed external state — but its forward intentions, which can still be modified. Steerability is not a property of the user interface. It is a property of the memory architecture. An agent designed from the start with prospective memory as a first-class primitive has structural support for steerability; in contrast, one for which steering is added afterward is steerable only by workaround.

Because every prospective commitment is interpreted by a model and persisted by a harness, AEI treats the model-harness interface contract as a first-class integrity surface: the contract must specify what epistemic state the harness supplies, what structured commitments the model must return, and which priority rules persist across model upgrades — a claim developed architecturally in the later Section 4.

2.4 The Stable What / Volatile How Principle

AEI is offered as a stable what — the invariant architectural obligation that persists regardless of implementation. For Knowing: the system must maintain non-contradictory beliefs and detect supersession. For Doing: it must track the cumulative state of capability invocations and reason about safe resumption. For Deciding: it must maintain an explicit history of intent states and surface changes for human review. The mechanisms that satisfy these obligations — the specific retrieval strategies, uncertainty scoring methods, capability logging schemas — are the volatile how. They will evolve as models improve and infrastructure matures. A system organized around the stable what survives that evolution with its core behavior intact. A system organized around today's specific mechanisms requires partial redesign each time those mechanisms change.

2.5 Four-Level Hierarchy

AEI then replaces flat ReAct-style action loops with a four-level hierarchy.

Level 1: Goal validity. The agent asks whether the goal is still valid. It reads the goal state, validity conditions, constraints, and update history. It produces a continuation decision. The relevant reward is not task completion alone. It is preservation of goal validity under constraints.

Level 2: archetype sequencing. An archetype is an abstract action class that groups tools by the operation they support, before a specific tool instance is chosen. This paper uses search, write, execute, navigate, and communicate as one practical archetype set. The set is not exhaustive, and can be extended as new tool families or agent settings require. At this level, the agent decides the next sub-goal and the action type needed to advance it. It selects an archetype first, then delegates concrete tool selection to Level 3. This separates "what kind of action is needed" from "which tool should perform it." This separation makes the model–harness boundary more stable. It also benefits training. The model can learn archetype selection independently from the changing tool registry, while later stages train tool-instance selection and invocation with separate reward signals.

Level 3: tool instance selection. The agent chooses the specific registered tool within the chosen archetype. This selection should be grounded in the tool registry. The registry must include an explicit archetype field. The model should not infer the archetype from the tool name alone. Its reward is selection accuracy and archetype consistency.

Level 4: invocation and failure discrimination. The agent synthesizes parameters, executes the tool, interprets output, and classifies failures. A parameter error calls for retrying the same tool with different parameters. An instance mismatch calls for a different tool within the same archetype. An archetype error calls for replanning at Level 2. A state error indicates an AEI violation and calls for escalation to Level 1. This final level is central. Many agents are trained to recover from failure. Fewer are trained to diagnose the kind of failure. AEI treats failure discrimination as its own training target. The reward should measure whether the model identified the right failure class, even if recovery later fails.

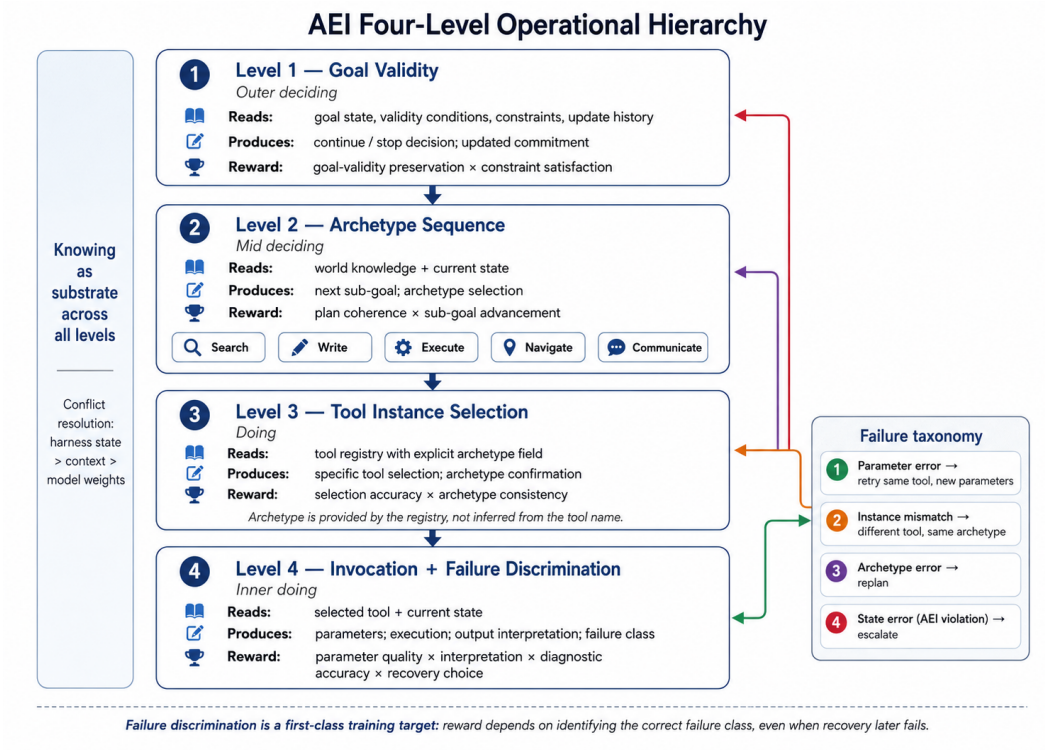

Figure 1: Four-level AEI operational hierarchy for long-running agents. Goal validity, archetype sequencing, tool instance selection, and invocation are treated as distinct control levels with separate reward signals. Failure discrimination is modeled as an explicit training target, with diagnostic classes determining whether recovery remains within the current tool, returns to tool selection, replans at the archetype level, or escalates to goal-validity revision.

This hierarchy maps naturally to the Options Framework, which formalizes temporally extended actions as options with initiation, execution, and termination conditions [12]. The mapping is intentional rather than fully formal: archetypes function analogously to options in initiation, internal execution, and termination, though precise correspondence in the Sutton-Precup-Singh sense is left to future work. It is initiated when a sub-goal requires a class of action; it is executed through tool selection and invocation; and it terminates when the sub-goal completes or when failure requires escalation. The contribution is not only hierarchy, but also hierarchy at the model-harness boundary. Tool use is not just a sequence of calls. It is a structured commitment that must remain interpretable across state transitions and model versions.

3 Evaluating Long-Running Epistemic Integrity

Current trajectory evaluations provide an important starting point for agent evaluation. They typically measure whether an agent can complete a task over an observed interaction trace, including intermediate reasoning, tool calls, planning choices, and recovery behavior. This makes them useful for testing bounded task competence. However, these evaluations do not fully test long-running epistemic integrity. AEI targets failures that are cross-session and cross-layer. Such failures emerge when state persists beyond the context window, when the harness carries records forward, when the model changes while the harness remains fixed, or when the tool registry evolves while active goals remain in force. A session-bounded benchmark may inspect the final answer, the trace, and even intermediate steps, yet still miss the relevant failure. What it often lacks is persistent epistemic state: state that ages, survives resets, crosses model versions, and re-enters later sessions with metadata. Without this structure, an agent can satisfy a trajectory-level objective while failing the conditions required for safe resumption. This limitation connects to recent work on denser supervision. Process-reward methods improve supervision over agent trajectories [14]. Memory-focused RL improves temporal

evidence selection in multi-session dialogue [3]. The latter is closest to AEI training, but remains primarily within the Knowing domain. AEI instead requires reward signals derived from interface obligations across all three domains: belief provenance in Knowing, capability-state resumption in Doing, and goal-validity preservation in Deciding.

These cross-session failures can appear in each AEI domain. In the Knowing domain, the harness may return the same memory record before and after a model upgrade, while the new model assigns it different weight relative to in-context evidence. The memory store has not failed, and the context is not malformed; the change occurs at the interpretation boundary. In the Doing domain, a tool schema may remain unchanged, while the model shifts its inferred archetype or invocation strategy. The executor still works, and the schema is still valid; the missing object is a stable contract between tool registration and model action. In the Deciding domain, a goal record may remain fixed, while the model changes how it interprets the goal, its constraints, or its validity conditions. The harness supplies the same state, but the agent's commitment changes.

AEI-aware evaluation therefore should not measure only bounded task success. It should measure whether persisted epistemic state remains stable, interpretable, and safe to resume. This paper proposes five evaluation dimensions. *Epistemic drift rate* measures how quickly the agent's belief state diverges from ground truth across session boundaries. *Resumption fidelity* measures whether capability state is sufficient for safe restart after interruption, including what has already been done, what remains, and what is safe to retry. *Replanning quality* measures whether revised plans remain coherent with the persisted goal record when conditions change. *Goal drift detection* measures sensitivity to goal expiration, constraint invalidation, or changed validity conditions across sessions. *Handoff fidelity* measures whether epistemic state is transferred completely when a trajectory crosses agents, sessions, or model versions.

These dimensions are not proposed as a final taxonomy. They are a minimum evaluation surface for making AEI violations observable. The key shift is from evaluating the session to evaluating the persisted trajectory. A persisted trajectory includes prior state, state updates, metadata, tool commitments, goal-validity conditions, and cross-session continuation. Under this view, an agent may complete a local task and still fail AEI if its beliefs are unsourced, its tool choices are not archetype-consistent, its goals are no longer valid, or its failure states cannot be diagnosed and resumed safely.

4 Model–Harness Interface Contracts

AEI requires explicit interface contracts across Knowing, Doing, and Deciding. These contracts specify what the harness must provide to the model, what the model must return to the harness, and which parts of the interaction must remain stable across model versions, context resets, and harness changes.

Knowing contract. The Knowing contract governs belief state. The harness must supply beliefs with provenance, including source, timestamp, validity period, and confidence signal. Content alone is insufficient. A fact without provenance cannot be safely revised; a memory without staleness metadata cannot be safely trusted; and a retrieved belief without source metadata becomes indistinguishable from ordinary context. In that case, harness-supplied evidence can collapse into the model's parametric priors precisely when it was meant to override them. The model must return conclusions with uncertainty and source attribution. It should identify which source supports a conclusion and distinguish retrieved evidence from model prior. This makes conflict resolution visible. The purpose is not only interpretability, but resumability: a later model version should be able to inspect the state and understand why a belief was held. This contract addresses Knowing-domain interface volatility by pinning the interpretive frame at the interface, rather than leaving belief weighting entirely to model inference.

Doing contract. The Doing contract governs tool use. The tool registry must include an explicit archetype field. A tool should not expose only a name, description, and parameter schema; it should also specify the abstract action class it implements. This removes archetype classification from model inference. The model must return semantic intent with each invocation. The invocation record should state why the tool was selected, what success would look like, and which strategy is being applied. This makes tool use stateful and capability state portable. A later model version should be able to

resume tool-related state without reconstructing the full plan from scratch. This contract addresses Doing-domain interface volatility by stabilizing the relation between tool registration and model action, even when the concrete tool inventory or model version changes.

Deciding contract. The Deciding contract governs goals and commitments. The harness must represent goals with validity conditions, constraint history, and update logs. A goal statement alone is insufficient. Long-running goals can expire, become unsafe, conflict with later instructions, or require revision when external conditions change. The model must return prospective commitments in a model-independent, harness-readable format. These commitments should survive context loss and model upgrades. Under this contract, a goal is not merely text to pursue. It is a stateful object with explicit validity conditions. This contract addresses Deciding-domain interface volatility by ensuring that goal interpretation depends on stable structure, rather than on a particular model's parsing habits.

Across all three domains, the same principle applies. The contract defines the stable *what*; the model may change the volatile *how*: the harness may change its implementation, and the model may change its internal reasoning, but the interface must remain stable enough for continuity.

This yields the paper's central claim: the interface contract is not an output of joint design; it is the precondition for joint design. Without such a contract, rewards cannot target the right behavior, evaluations cannot expose the right failures, harness architects cannot know what state must persist, and model trainers cannot know what outputs must be trained.

5 Contract-First Training: From Task Rewards to Interface Rewards

AEI changes the order of training design. A common pipeline is to build environments, define rewards, and then train models. This order optimizes what the environment already observes. It favors task completion, but does not guarantee interface integrity.

AEI requires a contract-first order. First, specify the model–harness interface contract. Second, derive rewards from the outputs that the contract requires. Third, build environments that generate the corresponding training signal. This ordering matters because many AEI-relevant outputs are not natural byproducts of task completion. A model can finish a task without source attribution. It can use a tool without recording semantic intent. It can continue a goal without checking validity. It can recover from failure without identifying the failure class. Such a model may score well under current objectives while still violating AEI.

The four-level hierarchy implies four reward families. Level 1 requires reward for goal-validity preservation and constraint satisfaction. Level 2 requires reward for coherent sub-goal and archetype sequencing. Level 3 requires reward for tool-selection accuracy and archetype consistency. Level 4 requires reward for parameter quality, output interpretation, failure classification, and recovery choice. The most distinctive signal is failure discrimination. Recovery success is not enough: the model must identify whether the failure is a parameter error, instance mismatch, archetype error, or state error. This requires labeled failure scenarios with diagnostic ground truth.

The four reward families use a natural decomposition under the Options Framework [12]. An archetype can be treated as an option. Level 2 selects which option to invoke, while Levels 3–4 define the option's intra-option execution. Within an option, the intra-option reward can be written as

$$r_{\text{intra}} = r_3 + r_4 + \mathbf{1}_{\text{fail}}\, r_4'.$$

Here, r_3 rewards tool-selection accuracy and archetype consistency, r_4 rewards parameter quality and output interpretation, and $r_{4'}$ rewards failure-class discrimination conditional on a failure occurring. The option-termination reward is assigned at Level 2 for sub-goal achievement and plan-coherence preservation. The high-level policy reward is assigned at Level 1 for goal-validity preservation across the trajectory.

This is not the only possible formalization. Its purpose is to make the training signal explicit. The four AEI levels correspond to distinct reward signals and distinct training-data requirements. Current trajectory-level rewards often collapse these distinctions into a single terminal or progress-based signal. The missing term is $r_{4'}$: failure-class discrimination requires labeled failure scenarios with diagnostic ground truth, which existing agent-training datasets generally do not provide.

AEI also separates two forms of tool-use training that are often merged. The first is selection training. It includes archetype classification and tool-instance retrieval. These are different problems: archetype

classification asks what kind of action is needed, while instance retrieval asks which registered tool best serves that action. The second is invocation training. It covers parameter synthesis, output interpretation, and continuation decisions. Treating tool use as a single action hides this structure and makes it harder to assign the right reward to the right failure.

Current simulation work addresses parts of the required infrastructure, but not the full AEI setting. Asynchronous environments such as ARE and Gaia2 introduce world state that evolves independently of agent action within a scenario, capturing intra-session temporal dynamics [1, 4]. LLM-based simulation such as Simia generates state transitions and tool feedback without real APIs, enabling scalable trajectory synthesis but making long-horizon consistency difficult [7]. Code-based synthesis such as Agent World Model produces deterministic, reproducible environments through programmatic state machines, improving reliability but limiting open-ended state-space coverage [13].

The remaining gap is cross-session persistence with aging harness state. Beliefs, capability records, and goal commitments established in one session must enter later sessions as persistent, possibly stale state. The model must then be trained to reason about provenance, staleness, and validity, rather than treating retrieved state as automatically current. Building such environments requires event-driven temporal simulation across session boundaries, combined with a persistent harness state layer. The interface contract specifies what this layer must contain. It is therefore not only an architectural requirement, but also a precondition for cross-session training infrastructure.

6 Related Work

This paper is closest to work on epistemic warrant, agent harnesses, tool use, hierarchical agent training, process rewards, uncertainty, and simulated agent environments. Across these areas, prior work improves components of agent reliability. AEI instead focuses on the contract that must hold across components over time.

Work on semantic laundering argues that agent architectures can mistake information transfer for epistemic justification: propositions may acquire apparent warrant merely by crossing trusted tool boundaries [10]. This paper shares the concern with warrant loss at architectural boundaries. It differs by focusing on long-running agents, cross-session persistence, and bidirectional model–harness contracts.

Recent harness work treats the agent harness as a first-class reliability determinant, rather than a thin execution wrapper [8]. This paper extends that view. It argues that the harness is not only a runtime substrate, but part of the epistemic architecture. The missing object is the model–harness interface contract.

Tool-use and agent-loop work shows that language models can interleave reasoning and acting, call external tools, and route computation through modular components [15, 11, 6]. These systems make tool use operational. They do not make the model–tool boundary epistemically stable across model upgrades, context resets, or persistent state changes.

Hierarchical reinforcement learning provides the formal basis for temporal abstraction through options [12]. Recent LLM-agent work applies hierarchy to long-horizon credit assignment by separating planning from execution [9]. This paper instead applies hierarchy to tool-use abstraction. Archetypes serve as options, while tool selection, invocation, and failure discrimination become lower-level execution problems.

Process-reward work improves supervision over agent trajectories [14]. Memory-focused RL improves temporal evidence selection in multi-session dialogue [3]. This is the closest existing work to AEI training, but it operates primarily within the Knowing domain, with retrieval correctness as the central reward signal. AEI requires rewards derived from interface obligations across all three domains, including capability-state resumption in Doing and goal-validity preservation in Deciding.

Simulation work has begun to support dynamic and scalable agent training. ARE and Gaia2 introduce asynchronous environments and verifiable action-level evaluation [5, 4]. Simia uses reasoning models to simulate environment feedback [7]. Agent World Model builds executable synthetic environments backed by structured state [13]. These works address intra-session dynamics and scalable synthesis. The remaining gap is cross-session persistence with aging harness state.

Finally, control- and uncertainty-oriented approaches address adjacent reliability problems. CAAF emphasizes deterministic harness enforcement [17]. AUQ turns uncertainty into inference-time control signals [16]. This paper treats these as complementary. Its target is epistemic integrity: stable belief, action, and commitment across the model–harness boundary.

7 Conclusion and Next Steps

This paper argues that long-running AI agents require epistemic integrity as a first-class architectural constraint. The central problem is not only model reliability. It is also interface stability. When the model and harness evolve independently, a change in one layer can produce failures that are visible only in the other. These failures resist local fixes because neither layer is wrong in isolation.

AEI reframes this problem as a joint design requirement. Knowing, Doing, and Deciding are not separate model subsystems. They are interdependent domains that must be stabilized through explicit model–harness contracts. These contracts define how beliefs, tools, goals, uncertainty, commitments, and failures are represented across time.

The main prescription is that the interface contract must precede training. Rewards should be derived from what the contract requires the model to produce. Environments should then be built to expose whether those outputs are correct, stable, and resumable. Without this ordering, agents can optimize task completion while violating the conditions needed for long-running operation.

The next step is empirical. AEI implies a new class of evaluations and training environments. These should test cross-session persistence, temporal aging, model upgrades, tool-registry changes, and goal-state revision. They should also label failure modes directly, so that models can be trained not only to recover, but to discriminate why recovery is needed.

A minimal testbed would combine three elements: persistent harness state, event-driven temporal change between sessions, and versioned model–harness interfaces. Such a testbed would make interface volatility observable. It would also make epistemic integrity trainable. The deeper point, however, is architectural rather than empirical. The interface contract is not an output of joint design; it is the precondition for joint design. Long-running agent capability does not emerge from independent improvements at either layer alone. It requires contracts that endure the evolution of both.

References

[1] Pierre Andrews, Amine Benhalloum, Gerard Moreno-Torres Bertran, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Romain Froger, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Grégoire Mialon, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Thomas Scialom, Vladislav Vorotilov, Mengjue Wang, and Ian Yu. ARE: Scaling up agent environments and evaluations. In *International Conference on Learning Representations*, 2026.

[2] Maria A. Brandimonte, Gilles O. Einstein, and Mark A. McDaniel, editors. *Prospective Memory: Theory and Applications*. Lawrence Erlbaum Associates, Mahwah, NJ, 1996.

[3] Yiming Du, Baojun Wang, Yifan Xiang, Zhaowei Wang, Wenyu Huang, Boyang Xue, Bin Liang, Xingshan Zeng, Fei Mi, Haoli Bai, Lifeng Shang, Jeff Z. Pan, Yuxin Jiang, and Kam-Fai Wong. Memory-T1: Reinforcement learning for temporal reasoning in multi-session agents, 2025.

[4] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Grégoire Mialon, and Thomas Scialom. Gaia2: Benchmarking llm agents on dynamic and asynchronous environments, 2026.

[5] Romain Froger et al. ARE: Scaling up agent environments and evaluations, 2025.

[6] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholtz. MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning, 2022.

[7] Yuetai Li, Huseyin A. Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. Simulating environments with reasoning models for agent training, 2025.

[8] Qianyu Meng, Yanan Wang, Liyi Chen, Yihang Li, Wei Wu, Wenyuan Jiang, Qimeng Wang, Chengqiang Lu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. Preprints.org, 2026. Version 3, posted April 28, 2026. Not peer reviewed.

[9] Jiangweizhi Peng, Yuanxin Liu, Ruida Zhou, Charles Fleming, Zhaoran Wang, Alfredo Garcia, and Mingyi Hong. HiPER: Hierarchical reinforcement learning with explicit credit assignment for large language model agents, 2026.

[10] Oleg Romanchuk and Roman Bondar. Semantic laundering in AI agent architectures: Why tool boundaries do not confer epistemic warrant, 2026.

[11] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[12] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and Semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

[13] Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning, 2026.

[14] Zhiheng Xi, Chenyang Liao, Guanyu Li, Yajie Yang, Wenxiang Chen, Zhihao Zhang, Binghai Wang, Senjie Jin, Yuhao Zhou, Jian Guan, Wei Wu, Tao Ji, Tao Gui, Qi Zhang, and Xuanjing Huang. AgentPRM: Process reward models for llm agents via step-wise promise and progress, 2025.

[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[16] Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Agentic uncertainty quantification, 2026.

[17] Tianbao Zhang. CAAF: Harness as an asset: Enforcing determinism via the convergent ai agent framework (CAAF), 2026.